

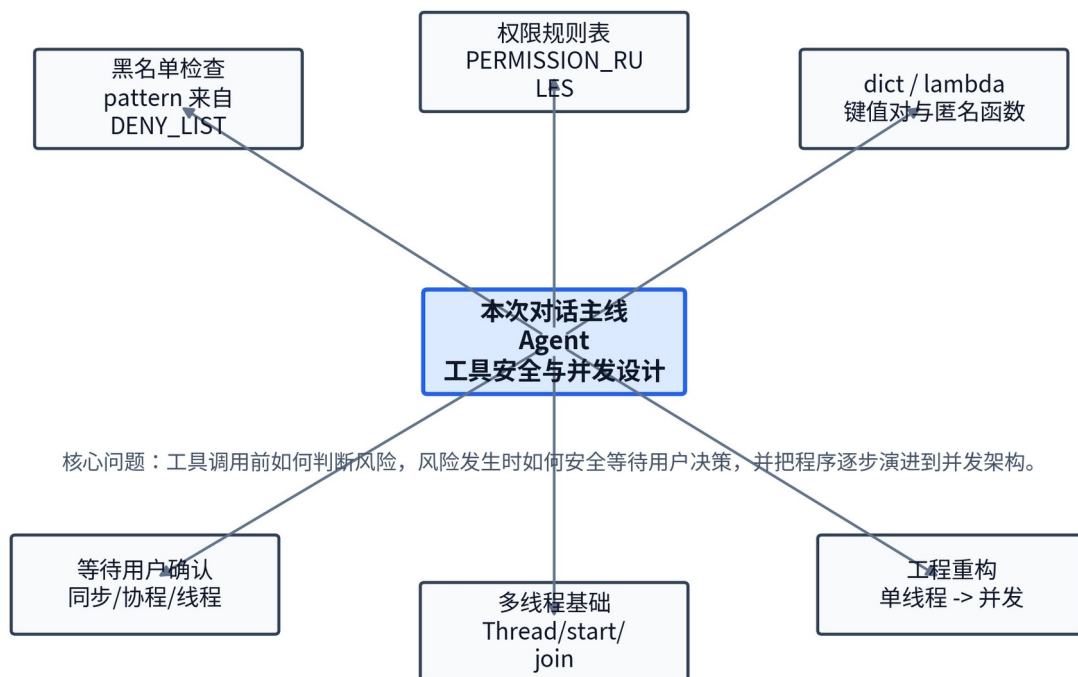
Agent 工具安全与并发设计学习笔记

基于本次对话整理：deny list、权限规则、dict/lambda、协程、多线程、并发重构

适用场景：正在学习 Agent Loop、工具调用、权限检查、用户确认、人机协作执行流程。

重点：本次对话的核心结论

Agent 的工具调用不能只关注“能不能执行”，还要在执行前做权限判断；当操作有风险时，应暂停工具调用并等待用户确认。并发不是一开始就全量改写，而是在结构清晰的单线程版本基础上，找到阻塞点和独立任务后局部重构。



目录

1. 从 DENY_LIST 到 pattern：黑名单检查的基本思想
2. PERMISSION_RULES：权限规则表逐层理解
3. dict 是什么：用 key 查 value 的容器
4. lambda 是什么：匿名函数不固定返回布尔值
5. 等待用户确认：同步、协程、多线程怎么选
6. Python 多线程基础：Thread、start、join、Queue、Lock
7. 工程实践：先单线程还是直接并发？如何重构
8. Agent 项目推荐架构与速查表

1. 从 DENY_LIST 到 pattern：黑名单检查的基本思想

本次最开始讨论的代码是：

```
def check_deny_list(command: str) -> str | None:
    for pattern in DENY_LIST:
        if pattern in command:
            return f"Error: Command '{command}' is not allowed"
    return None
```

重点：pattern 不是特殊语法

pattern 是 for 循环里的临时变量。它每次从 DENY_LIST 中取出一个元素，用来检查这个危险关键词是否出现在 command 字符串里。

代码片段	含义
for pattern in DENY_LIST	遍历黑名单列表，每次取出一个危险关键词，临时命名为 pattern。
if pattern in command	判断这个关键词是否包含在用户输入的命令字符串中。
return Error...	只要命中一个危险关键词，就立即返回错误，不继续执行。
return None	所有规则都没命中，说明没有发现黑名单风险。

如果：

```
DENY_LIST = ["rm -rf", "sudo", "shutdown"]
command = "sudo apt install vim"
```

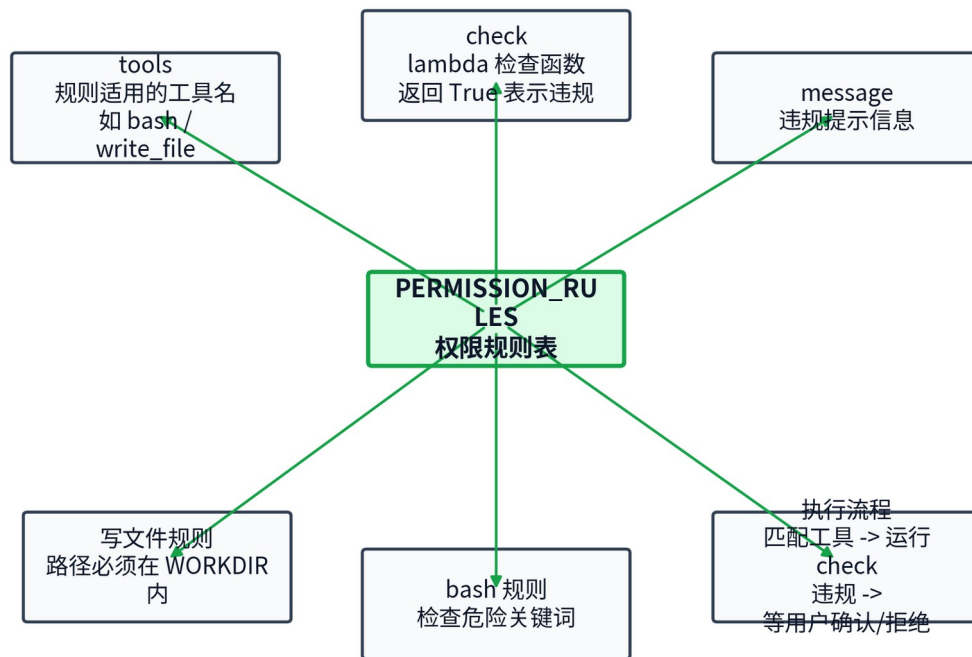
循环过程可以想成：

```
pattern = "rm -rf"    # False
pattern = "sudo"      # True, 命中，返回错误
pattern = "shutdown"  # 不会继续检查，因为已经 return
```

2. PERMISSION_RULES：权限规则表逐层理解

后面讨论的权限规则代码是：

```
PERMISSION_RULES = [
    {"tools": ["write_file", "edit_file"],
      "check": lambda args: not (WORKDIR / args.get("path", "")).resolve().is_relative_to(WORKDIR),
      "message": "Writing outside workspace"},
    {"tools": ["bash"],
      "check": lambda args: any(kw in args.get("command", "") for kw in ["rm ", "> /etc/", "chmod 777"]),
      "message": "Potentially destructive command"},
]
```



重点：这不是“执行逻辑”，而是“规则配置”

PERMISSION_RULES 是一个 list，里面每个元素是一个 dict。每个 dict 描述一条权限规则：哪些工具适用、如何检查、违规时提示什么。

字段	类型	作用
tools	list[str]	这条规则适用于哪些工具，例如 bash、write_file、edit_file。
check	function	检查函数，接收 args，返回 True/False。通常 True 表示触发风险。
message	str	触发规则时给用户看的提示信息。

2.1 写文件规则：阻止写出工作区

```
"check": lambda args: not (WORKDIR / args.get("path", "")).resolve().is_relative_to(WORKDIR)
```

片段	解释
args.get("path", "")	从工具参数里取 path；没有 path 就返回空字符串。
WORKDIR / path	使用 pathlib 拼接工作目录和目标路径。这里的 / 是路径拼接，不是除法。
.resolve()	解析成标准绝对路径，消除 ../ 等路径跳转。
.is_relative_to(WORKDIR)	判断最终路径是否仍在工作目录下。

not (...)	取反：不在工作目录内就返回 True，表示违规。
-----------	--------------------------

重点：为什么要 resolve()

用户可能传入 ../secret.txt。表面上它是 WORKDIR/ ../secret.txt，实际解析后可能跑到工作区外。resolve() 用来识别这种路径逃逸。

2.2 bash 规则：检查潜在破坏性命令

```
"check": lambda args: any(
    kw in args.get("command", "")
    for kw in ["rm ", "> /etc/", "chmod 777"]
)
```

片段	解释
args.get("command", "")	从工具参数中取出要执行的 shell 命令。
for kw in [...]	依次取出危险关键词，例如 rm、写入 /etc、chmod 777。
kw in command	判断命令字符串中是否包含这个关键词。
any(...)	只要任意一个关键词命中，就返回 True。

2.3 实际使用方式

```
def check_permission(tool_name: str, args: dict) -> str | None:
    for rule in PERMISSION_RULES:
        if tool_name in rule["tools"]:
            if rule["check"](args):
                return rule["message"]
    return None
```

重点：工程意义

这相当于 Agent 工具调用前的安全拦截器：先看工具名是否匹配某条规则，再执行 check(args)，如果触发风险，就不要直接执行工具。

3. dict 是什么：用 key 查 value 的容器

dict 是 Python 的字典类型，用来保存键值对。

```
student = {
    "name": "Tom",
    "age": 18,
    "city": "Singapore"
}

print(student["name"]) # Tom
```

概念	含义	例子
key	键，用来查找值	"name"、"age"、"tools"
value	值，key 对应的数据	"Tom"、18、["bash"]
dict	键值对容器	{"tools": ["bash"], "message": "..."}

重点：list 和 dict 的区别

list 用下标找值，例如 arr[0]；dict 用 key 找值，例如 rule["tools"]。权限规则很适合用 dict，因为每条规则都有固定字段：tools、check、message。

4. lambda 是什么：匿名函数不固定返回布尔值

lambda 是 Python 的匿名函数，格式是：

```
lambda 参数：表达式
```

它自动返回冒号后面表达式的结果。返回什么类型，取决于这个表达式。

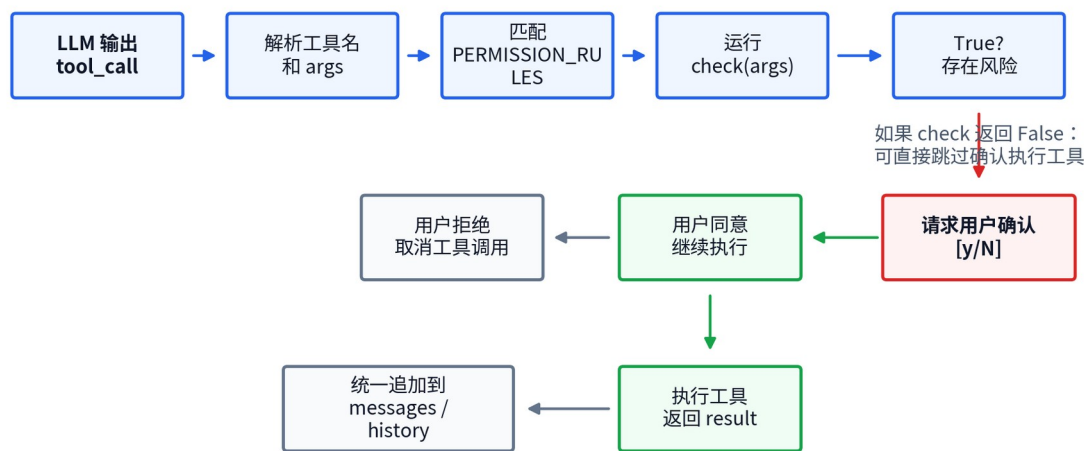
lambda 示例	返回类型	说明
lambda x: x > 10	bool	表达式是比较判断，所以返回 True/False。
lambda x: x + 1	int/float	表达式是加法，返回数字。
lambda name: "hello " + name	str	表达式是字符串拼接，返回字符串。
lambda x: [x, x * 2]	list	表达式构造列表，返回 list。

重点：你这段代码中的 lambda 为什么返回布尔值

因为 check 后面的表达式是 not(...) 或 any(...)，这两类表达式都会产生 True/False。所以不是 lambda 固定返回 bool，而是这段权限检查刚好返回 bool。

可以把 lambda 理解成只能写一个表达式的短函数；复杂逻辑建议改写成 def，便于调试和加断点。

5. 等待用户确认：同步、协程、多线程怎么选



重点：等待用户确认是 Agent 安全系统的关键环节

当权限检查发现风险时，程序应该暂停工具执行，向用户请求确认。用户同意才继续，用户拒绝则取消。这一步可以同步写，也可以用协程或线程写。

方案	写法	适合场景	缺点
同步 input()	answer = input(...)	终端版教学 demo，逻辑最简单	会阻塞整个程序
协程 async/await	approved = await ask_user_confirm(...)	Web UI、WebSocket、流式输出、多会话 Agent	需要事件循环和异步接口
多线程 threading	Thread(target=ask_user_confirm)	阻塞函数 input、subprocess、文件 I/O 等	需要注意共享状态和线程通信

5.1 同步版：最适合初学阶段

```
def ask_user_confirm(message: str) -> bool:
    answer = input(f"{message}. Continue? [y/N]: ")
    return answer.lower() == "y"
```

优点是简单；缺点是 input() 会卡住整个程序，直到用户输入。

5.2 协程版：适合异步 Agent

```
import asyncio

async def ask_user_confirm(message: str) -> bool:
    answer = await asyncio.to_thread(
```

```

    input,
    f"{message}. Continue? [y/N]: "
)
return answer.lower() == "y"

```

这里用 `asyncio.to_thread` 包装 `input`，是因为 `input` 本身是阻塞函数。这样不会直接堵死事件循环。

5.3 工具执行前的统一封装

```

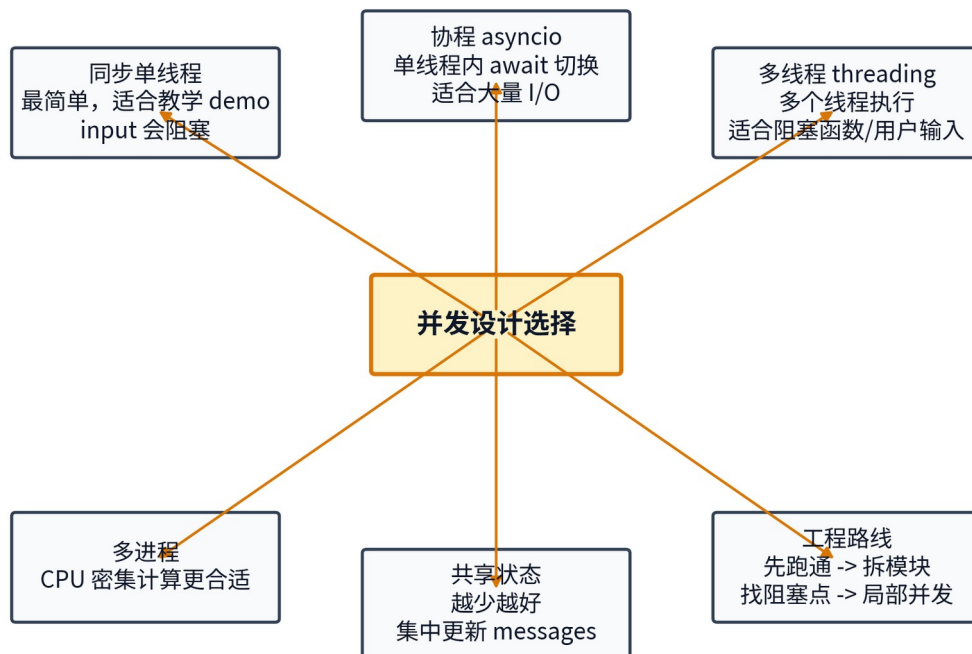
async def run_tool_with_permission(tool_name: str, args: dict):
    warning = check_permission(tool_name, args)

    if warning:
        approved = await ask_user_confirm(warning)
        if not approved:
            return "Tool call cancelled by user"

    return await run_tool(tool_name, args)

```

6. Python 多线程基础：Thread、start、join、Queue、Lock



重点：协程不是多线程

多线程是真的创建多个线程，由操作系统调度；协程通常是在一个线程里，任务执行到 `await` 时主动让出控制权。它们都能实现并发，但机制不同。

6.1 最小多线程示例

```
import threading
import time

def worker():
    for i in range(5):
        print("worker running", i)
        time.sleep(1)

t = threading.Thread(target=worker)
t.start()
print("main thread running")
t.join()
print("done")
```

写法	作用
threading.Thread(target=worker)	创建线程对象，指定线程启动后执行 worker 函数。
t.start()	真正启动线程。
t.join()	主线程等待子线程执行完。
target=worker	传函数本身，不要写成 target=worker()。

6.2 多线程等待用户确认

```
import threading
import queue

def ask_user_confirm(message: str, q: queue.Queue):
    answer = input(f"{message}. Continue? [y/N]: ")
    q.put(answer.lower() == "y")

def run_tool_with_permission(tool_name: str, args: dict):
    warning = check_permission(tool_name, args)

    if warning:
        q = queue.Queue()
        t = threading.Thread(target=ask_user_confirm, args=(warning, q))
        t.start()
        approved = q.get()
        t.join()

        if not approved:
            return "Tool call cancelled by user"

    return run_tool(tool_name, args)
```

重点：为什么推荐 Queue

多个线程之间传递结果时，Queue 比共享 dict 更稳。它更像“线程之间传消息”，能减少共享变量带来的混

乱。

6.3 共享变量与 Lock

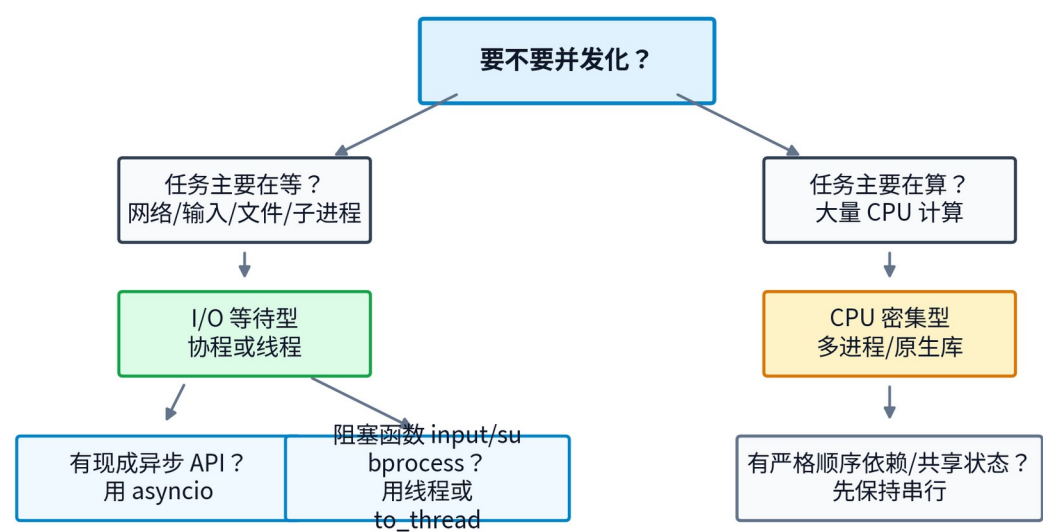
如果多个线程同时修改同一个变量，就可能出现竞争条件。用 Lock 可以保证同一时间只有一个线程进入临界区。

```
import threading

counter = 0
lock = threading.Lock()

def add():
    global counter
    for _ in range(100000):
        with lock:
            counter += 1
```

7. 工程实践：先单线程还是直接并发？如何重构



重点：工程上的推荐路线

先把业务逻辑按“可并发的结构”拆清楚，第一版可以先用单线程跑通；等功能正确并定位到阻塞点后，再把局部重构成协程、线程或进程。

阶段	目标	不要做什么
第一版单线程	跑通主流程：LLM -> tool_use -> tool_result -> history	不要一开始就到处创建线程。
模块拆分	拆出 check_permission、	不要把所有逻辑堆在一个 while 里。

	ask_user_confirm、run_tool、append_result	
找阻塞点	识别 input、LLM API、网络请求、subprocess、文件 I/O	不要凭感觉并发化所有函数。
局部并发	只并发等待型、独立型、无共享状态的任务	不要并发执行有顺序依赖的写文件/改文件任务。
状态集中更新	工具结果收集完后统一更新 messages/history	不要让多个线程同时 append messages。

7.1 判断用线程、协程还是多进程

任务类型	推荐方式	例子
等待 I/O	协程或线程	网络请求、用户输入、文件读写、子进程等待
阻塞函数	线程或 <code>asyncio.to_thread</code>	<code>input()</code> 、某些同步 SDK、 <code>subprocess.run</code>
大量异步网络任务	<code>asyncio</code>	WebSocket、HTTP API、多工具远程调用
CPU 密集计算	多进程或底层释放 GIL 的库	图像处理、模型推理、复杂计算
强顺序依赖	保持串行	先写文件，再编辑，再运行

重点：判断口诀

如果任务在“等”，考虑协程或线程；如果任务在“算”，考虑多进程；如果任务有先后依赖，保持串行；如果任务共享状态，先别急着并发。

7.2 单线程版本如何重构成并发版本

- 9. 先把大函数拆成小函数，明确每个函数只负责一件事。
- 10. 把权限检查做成纯函数：输入 `tool_name` 和 `args`，输出 `warning` 或 `None`。
- 11. 把用户确认封装成 `ask_user_confirm`，不要和工具执行混在一起。
- 12. 把工具执行封装成 `run_tool`，让它只返回 `result`，不直接修改全局 `messages`。
- 13. 如果有多个独立工具，先并发执行工具，再统一收集结果。
- 14. 最后由主循环统一更新 `messages/history`，避免并发修改核心状态。

7.3 Agent 并发化伪代码

```
async def handle_tool_call(tool_call: dict):
    tool_name = tool_call["name"]
    args = tool_call["args"]
```

```

warning = check_permission(tool_name, args)
if warning:
    approved = await ask_user_confirm(warning)
    if not approved:
        return {"tool_name": tool_name, "result": "cancelled by user"}

result = await run_tool(tool_name, args)
return {"tool_name": tool_name, "result": result}

async def agent_loop():
    messages = []

    while True:
        response = await call_llm(messages)

        if not response["tool_calls"]:
            return response["content"]

        tasks = [handle_tool_call(tc) for tc in response["tool_calls"]]
        tool_results = await asyncio.gather(*tasks)

        # 核心状态集中更新，避免多个任务同时改 messages
        for result in tool_results:
            messages.append(result)

```

8. Agent 项目推荐架构与速查表

重点：推荐架构

Agent Loop 负责主流程；权限模块只判断风险；用户交互模块只负责确认；工具模块只执行工具；消息模块统一更新 history。边界清晰之后，才容易安全地并发化。

模块	职责	是否适合并发
agent_loop	驱动 LLM -> tool -> result 循环	核心控制流通常保持可控串行
check_permission	匹配权限规则，判断是否危险	纯计算，通常无需并发
ask_user_confirm	等待用户输入或前端按钮确认	适合协程/线程
run_tool	执行 bash、读写文件、网络请求等	I/O 型工具适合并发
collect_results	收集多个工具返回值	适合 asyncio.gather 等方式
update_messages	更新 messages/history	建议集中串行处理

8.1 本次对话知识点速查

问题	答案
pattern 哪来的？	来自 for pattern in DENY_LIST，是循环临时变量。
dict 是什么？	键值对容器，用 key 查 value。

lambda 一定返回 bool 吗？	不一定，返回冒号后表达式的结果；你这里因为表达式是 any/not，所以返回 bool。
协程是不是多线程？	不是。协程通常是单线程内通过 await 协作切换；多线程是真正创建多个线程。
等待用户确认适合协程吗？	适合，尤其是 Web UI、流式输出、多会话 Agent；终端 demo 同步 input 也可以。
工程上先并发还是先单线程？	设计时考虑并发边界，第一版可先单线程跑通，再局部重构。

8.2 最终记忆版

重点：一句话总结

Agent 工具调用的安全设计，本质是“规则判断 + 用户确认 + 可控执行”；并发设计的本质是“把等待型、独立型任务拆出去，同时把核心状态更新集中回来”。